

Summary

It's easy to program for years and still be unfamiliar with certain programming terms. But most major software applications are created by teams of software developers, not individuals. So being able to communicate unambiguously is important when you're working with a team.

This chapter explained that Python programs are made up of identifiers, variables, literals, keywords, and objects, and that all Python objects have a value, data type, and identity. Although every object has a data type, there are also several broad categories of types, such as container, sequence, mapping, set, built-in, and user-defined.

Some terms, like values, variables, and functions, have different names in specific contexts, such as items, parameters, arguments, and methods. Several terms are also easy to confuse with each other. It's not a big deal to confuse some of these terms in day-to-day programming: for example, property versus attribute, block versus body, exception versus error, or the subtle differences between library, framework, SDK, engine, and API. Other misunderstandings won't make the code you write wrong but might make you look unprofessional: for example, statement and expression, function and method, and parameter and argument are commonly used interchangeably by beginners.

But other terms, such as iterable versus iterator, syntax error versus semantic error, and bytecode versus machine code, have distinct meanings that you should never confuse with each other unless you want to confuse your colleagues.

You'll still find that the use of terms varies from language to language and even programmer to programmer. You'll become more familiar with jargon with experience (and frequent web searches) in time.

Further Reading

The official Python glossary at <https://docs.python.org/3/glossary.html> lists short but helpful definitions the Python ecosystem uses. The official Python